# i have a python flex app . which has structure like this . i want you to write me a bat file which will install docker and make a container for this app with all dependencies in docker , so i can use it in another computer and environment .

```
├──static
├──templates
├──templates - Copy
├──temp_cache
├──venv
│ ├──Include
│ ├──Lib
│ │ └──site-packages
│ │ ├──blinker
│ │ │ └──__pycache__
│ │ ├──blinker-1.9.0.dist-info
│ │ ├──cffi
│ │ │ └──__pycache__
│ │ ├──cffi-2.0.0.dist-info
│ │ │ └──licenses
│ │ ├──click
│ │ │ └──__pycache__
│ │ ├──click-8.3.1.dist-info
│ │ │ └──licenses
│ │ ├──colorama
│ │ │ ├──tests
│ │ │ │ └──__pycache__
│ │ │ └──__pycache__
│ │ ├──colorama-0.4.6.dist-info
│ │ │ └──licenses
│ │ ├──cryptography
│ │ │ ├──hazmat
│ │ │ │ ├──asn1
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──backends
│ │ │ │ │ ├──openssl
│ │ │ │ │ │ └──__pycache__
```

```
│ │ │ │ │ └──── __pycache__
│ │ │ │ ├──── bindings
│ │ │ │ │ ├──── openssl
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ ├──── _rust
│ │ │ │ │ │ └──── openssl
│ │ │ │ │ └──── __pycache__
│ │ │ │ ├──── decrepit
│ │ │ │ │ ├──── ciphers
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ └──── __pycache__
│ │ │ │ ├──── primitives
│ │ │ │ │ ├──── asymmetric
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ ├──── ciphers
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ ├──── kdf
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ ├──── serialization
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ ├──── twofactor
│ │ │ │ │ │ └──── __pycache__
│ │ │ │ │ └──── __pycache__
│ │ │ │ └──── __pycache__
│ │ │ ├──── x509
│ │ │ │ └──── __pycache__
│ │ │ └──── __pycache__
│ │ ├──── cryptography-46.0.5.dist-info
│ │ │ └──── licenses
│ │ ├──── flask
│ │ │ ├──── json
│ │ │ │ └──── __pycache__
│ │ │ ├──── sansio
│ │ │ │ └──── __pycache__
│ │ │ └──── __pycache__
│ │ ├──── flask-3.1.3.dist-info
│ │ │ └──── licenses
│ │ ├──── itsdangerous
│ │ │ └──── __pycache__
│ │ ├──── itsdangerous-2.2.0.dist-info
│ │ ├──── jinja2
│ │ │ └──── __pycache__
│ │ ├──── jinja2-3.1.6.dist-info
│ │ │ └──── licenses
│ │ ├──── markupsafe
│ │ │ └──── __pycache__
```

```
│ │ ├──markupsafe-3.0.3.dist-info
│ │ │ └──licenses
│ │ ├──oracledb
│ │ │ ├──plugins
│ │ │ │ └──__pycache__
│ │ │ └──__pycache__
│ │ ├──oracledb-3.4.2.dist-info
│ │ │ └──licenses
│ │ ├──PIL
│ │ │ └──__pycache__
│ │ ├──pillow-12.1.1.dist-info
│ │ │ └──licenses
│ │ ├──pip
│ │ │ ├──_internal
│ │ │ │ ├──cli
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──commands
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──distributions
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──index
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──locations
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──metadata
│ │ │ │ │ ├──importlib
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──models
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──network
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──operations
│ │ │ │ │ ├──build
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──install
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──req
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──resolution
│ │ │ │ │ ├──legacy
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──resolvelib
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
```

```
│ │ │ │ ├──utils
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──vcs
│ │ │ │ │ └──__pycache__
│ │ │ │ └──__pycache__
│ │ │ ├──_vendor
│ │ │ │ ├──cachecontrol
│ │ │ │ │ ├──caches
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──certifi
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──dependency_groups
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──distlib
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──distro
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──idna
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──msgpack
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──packaging
│ │ │ │ │ ├──licenses
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──pkg_resources
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──platformdirs
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──pygments
│ │ │ │ │ ├──filters
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──formatters
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──lexers
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──styles
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──pyproject_hooks
│ │ │ │ │ ├──_in_process
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──requests
│ │ │ │ │ └──__pycache__
```

```
│ │ │ │ ├──resolvelib
│ │ │ │ │ ├──resolvers
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──rich
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──tomli
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──tomli_w
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──truststore
│ │ │ │ │ └──__pycache__
│ │ │ │ ├──urllib3
│ │ │ │ │ ├──contrib
│ │ │ │ │ │ ├──_securetransport
│ │ │ │ │ │ │ └──__pycache__
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──packages
│ │ │ │ │ │ ├──backports
│ │ │ │ │ │ │ └──__pycache__
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ ├──util
│ │ │ │ │ │ └──__pycache__
│ │ │ │ │ └──__pycache__
│ │ │ │ └──__pycache__
│ │ │ └──__pycache__
│ │ ├──pip-25.2.dist-info
│ │ │ └──licenses
│ │ │ └──src
│ │ │ └──pip
│ │ │ └──_vendor
│ │ │ ├──cachecontrol
│ │ │ ├──certifi
│ │ │ ├──dependency_groups
│ │ │ ├──distlib
│ │ │ ├──distro
│ │ │ ├──idna
│ │ │ ├──msgpack
│ │ │ ├──packaging
│ │ │ ├──pkg_resources
│ │ │ ├──platformdirs
│ │ │ ├──pygments
│ │ │ ├──pyproject_hooks
│ │ │ ├──requests
│ │ │ ├──resolvelib
│ │ │ ├──rich
```

```
│ │ │ ├──tomli
│ │ │ ├──tomli_w
│ │ │ ├──truststore
│ │ │ └──urllib3
│ │ ├──pycparser
│ │ │ └──__pycache__
│ │ ├──pycparser-3.0.dist-info
│ │ │ └──licenses
│ │ ├──typing_extensions-4.15.0.dist-info
│ │ │ └──licenses
│ │ ├──werkzeug
│ │ │ ├──datastructures
│ │ │ │ └──__pycache__
│ │ │ ├──debug
│ │ │ │ ├──shared
│ │ │ │ └──__pycache__
│ │ │ ├──middleware
│ │ │ │ └──__pycache__
│ │ │ ├──routing
│ │ │ │ └──__pycache__
│ │ │ ├──sansio
│ │ │ │ └──__pycache__
│ │ │ ├──wrappers
│ │ │ │ └──__pycache__
│ │ │ └──__pycache__
│ │ ├──werkzeug-3.1.5.dist-info
│ │ │ └──licenses
│ │ └──__pycache__
│ └──Scripts
└──__pycache__
```

I now have the complete picture — your app uses **Oracle DB** (`oracledb`, thin mode, no Instant Client needed), **Flask**, **Pillow**, connects to `103.113.200.20:1521/nuorcl`, runs on Windows, and the entry point is `tabulation_web.py`. Here are all 3 files you need:

### ⊠ File 1 — `Dockerfile`

Place this in your project root (same folder as `tabulation_web.py`):

```
# ────────────────────────────────────────────────
#  NU Tabulation Archive  -  Docker Image
#  Python 3.11 slim  |  oracledb thin mode (no IC)
# ────────────────────────────────────────────────
FROM python:3.11-slim

# System deps for Pillow (JPEG support)
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```
    libjpeg-turbo-progs \
    libpng-dev \
    libfreetype6-dev \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Working directory inside container
WORKDIR /app

# Copy requirements first for layer caching
COPY requirements.txt .

# Install all Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the entire project (excluding venv via .dockerignore)
COPY . .

# Create temp_cache directory
RUN mkdir -p /app/temp_cache

# Expose Flask port
EXPOSE 5000

# Run with Gunicorn for production (threaded)
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--threads", "4", "--time
```

## File 2 — `requirements.txt`

Place in project root:

```
flask==3.1.3
oracledb==3.4.2
pillow==12.1.1
gunicorn==23.0.0
cryptography==46.0.5
werkzeug==3.1.5
```

## File 3 — `.dockerignore`

Place in project root to keep image lean:

```
venv/
__pycache__/
*.pyc
*.pyo
temp_cache/
templates - Copy/
*.bat
```

```
.git/
.gitignore
```

### File 4 — `docker_deploy.bat`

This is your **one-click bat file** — run on any Windows machine:

```
@echo off
title NU Tabulation Archive - Docker Deploy
color 0B
cls

echo.
echo ═══════════════════════════════════════════════════════════════
echo  NU TABULATION ARCHIVE  -  Docker Deploy Script
echo  Installs Docker Desktop (if needed) + Builds + Runs Container
echo ═══════════════════════════════════════════════════════════════
echo.

:: ── CONFIGURATION ──────────────────────────────────────────────
set IMAGE_NAME=nu-tabulation
set CONTAINER_NAME=nu-tabulation-app
set HOST_PORT=5000
set CONTAINER_PORT=5000
set PROJECT_DIR=%~dp0
:: ───────────────────────────────────────────────────────────────

echo [INFO] Project directory: %PROJECT_DIR%
echo.

:: ═══════════════════════════════════════════════════════════════
:: STEP 1 — CHECK / INSTALL DOCKER
:: ═══════════════════════════════════════════════════════════════
echo [1/5] Checking Docker installation...
docker --version >nul 2>&1
if %errorlevel% EQU 0 (
    echo [OK] Docker is already installed.
    docker --version
    goto :check_running
)

echo [!] Docker not found. Starting automatic installation...
echo.

:: Check if winget is available (Windows 10/11)
winget --version >nul 2>&1
if %errorlevel% EQU 0 (
    echo [INFO] Installing Docker Desktop via winget...
    winget install -e --id Docker.DockerDesktop --silent --accept-package-agreements --ac
    if %errorlevel% NEQ 0 (
        goto :manual_install
    )
    echo [OK] Docker Desktop installed via winget.
    goto :wait_docker
```

```
)

:manual_install
echo [INFO] winget not available. Downloading Docker Desktop installer...
echo.
set DOCKER_INSTALLER=%TEMP%\DockerDesktopInstaller.exe
curl -L "https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe" -o
if %errorlevel% NEQ 0 (
    echo [ERROR] Download failed. Please install Docker Desktop manually:
    echo         https://www.docker.com/products/docker-desktop/
    pause
    exit /b 1
)
echo [INFO] Running Docker Desktop installer (follow the GUI)...
"%DOCKER_INSTALLER%" install --quiet
echo [OK] Docker Desktop installed.

:wait_docker
echo.
echo [INFO] Docker Desktop needs to start. Waiting 45 seconds...
echo        (If it prompts for WSL2 update, approve it)
timeout /t 45 /nobreak >nul

:: Try to start Docker service
net start com.docker.service >nul 2>&1
timeout /t 15 /nobreak >nul

:: Verify docker is now responsive
docker --version >nul 2>&1
if %errorlevel% NEQ 0 (
    echo.
    echo [!] Docker is installed but not yet running.
    echo     Please:
    echo       1. Launch "Docker Desktop" from the Start Menu
    echo       2. Wait for the whale icon in the taskbar to stop animating
    echo       3. Re-run this script
    echo.
    pause
    exit /b 1
)
echo [OK] Docker is running.

:check_running
:: ═══════════════════════════════════════════════════════════════════════
:: STEP 2 — ENSURE DOCKER DAEMON IS RUNNING
:: ═══════════════════════════════════════════════════════════════════════
echo.
echo [2/5] Checking Docker daemon status...
docker info >nul 2>&1
if %errorlevel% NEQ 0 (
    echo [!] Docker daemon is not running. Attempting to start Docker Desktop...
    start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"
    echo [INFO] Waiting 40 seconds for Docker to start...
    timeout /t 40 /nobreak >nul
    docker info >nul 2>&1
    if %errorlevel% NEQ 0 (
```

```
        echo [ERROR] Docker daemon still not responding.
        echo         Please start Docker Desktop manually and re-run this script.
        pause
        exit /b 1
    )
)
echo [OK] Docker daemon is running.

:: ═══════════════════════════════════════════════════════════════════════
:: STEP 3 — STOP & REMOVE OLD CONTAINER (if exists)
:: ═══════════════════════════════════════════════════════════════════════
echo.
echo [3/5] Cleaning up old container (if any)...
docker stop %CONTAINER_NAME% >nul 2>&1
docker rm   %CONTAINER_NAME% >nul 2>&1
echo [OK] Old container removed (or did not exist).

:: ═══════════════════════════════════════════════════════════════════════
:: STEP 4 — BUILD IMAGE
:: ═══════════════════════════════════════════════════════════════════════
echo.
echo [4/5] Building Docker image: %IMAGE_NAME%
echo       (This may take 2-5 minutes on first run — downloading layers)
echo.
cd /d "%PROJECT_DIR%"
docker build -t %IMAGE_NAME% .
if %errorlevel% NEQ 0 (
    echo.
    echo [ERROR] Docker build failed!
    echo         Common causes:
    echo           - Missing requirements.txt
    echo           - Missing Dockerfile
    echo           - No internet connection during first build
    echo.
    pause
    exit /b 1
)
echo.
echo [OK] Image built successfully: %IMAGE_NAME%

:: ═══════════════════════════════════════════════════════════════════════
:: STEP 5 — RUN CONTAINER
:: ═══════════════════════════════════════════════════════════════════════
echo.
echo [5/5] Starting container: %CONTAINER_NAME%
echo       Port mapping: localhost:%HOST_PORT% → container:%CONTAINER_PORT%
echo.

docker run -d ^
    --name %CONTAINER_NAME% ^
    --restart unless-stopped ^
    -p %HOST_PORT%:%CONTAINER_PORT% ^
    -v "%PROJECT_DIR%temp_cache:/app/temp_cache" ^
    %IMAGE_NAME%

if %errorlevel% NEQ 0 (
```

```
    echo [ERROR] Failed to start container.
    docker logs %CONTAINER_NAME%
    pause
    exit /b 1
)

:: ═══════════════════════════════════════════════════════════════════════
:: SUCCESS
:: ═══════════════════════════════════════════════════════════════════════
echo.
echo ═══════════════════════════════════════════════════════════
echo  SUCCESS! Container is running.
echo ═══════════════════════════════════════════════════════════
echo.
echo  App URL  :  http://localhost:%HOST_PORT%
echo  Container:  %CONTAINER_NAME%
echo  Image    :  %IMAGE_NAME%
echo.
echo  Useful commands:
echo    View logs   :  docker logs -f %CONTAINER_NAME%
echo    Stop app    :  docker stop %CONTAINER_NAME%
echo    Restart app :  docker restart %CONTAINER_NAME%
echo    Shell inside:  docker exec -it %CONTAINER_NAME% bash
echo    Remove all  :  docker stop %CONTAINER_NAME% ^& docker rm %CONTAINER_NAME%
echo.

:: Auto-open browser after 3 seconds
echo [INFO] Opening browser in 3 seconds...
timeout /t 3 /nobreak >nul
start http://localhost:%HOST_PORT%

echo.
echo Press any key to exit this window (container keeps running).
pause >nul
```

## How to Use

**On your development machine:**

1. Place `Dockerfile`, `requirements.txt`, `.dockerignore`, and `docker_deploy.bat` in your project root
2. Run `docker_deploy.bat` as **Administrator**

**To copy to another computer:**
Copy your **entire project folder** (including these 4 new files, excluding `venv/`) to the new machine and run `docker_deploy.bat` — it handles everything automatically.

## What the BAT Does — Step by Step

| Step | Action |
|---|---|
| 1 | Checks if Docker is installed; if not, installs via `winget` or downloads installer |
| 2 | Waits for Docker daemon to be ready |
| 3 | Removes any old container with the same name |
| 4 | Builds a fresh Docker image from your `Dockerfile` |
| 5 | Runs the container with port `5000` exposed and `temp_cache` mounted as a volume |

> **Important:** The `temp_cache` folder is mounted as a **volume** (`-v`), so your disk-cached images persist across container restarts and are not lost when you rebuild the image.

⁂

1. cache.py
2. login_final_1.html
3. login.html
4. login_final_3.html
5. login_final2.html
6. db.py
7. T_USER.sql
8. tabulation_web.py
9. app.py
10. start.bat
11. admin.html
12. index.html
13. index-Copy.html